

0.15.2. Deux rythmes, un seul chronomètre — doublé, deux positions

Les deux rythmes à deux places : en colonne 0 et en colonne 10.

1. Ce qu'il faut comprendre

C'est le 0.15 et le 0.15.1 réunis : les deux `poser` du 0.15, et les deux du 0.15.1.

Quatre `poser`, deux variables : la lente est la même en colonne 0 et en colonne 10, la rapide aussi.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **974 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	58 o	← la console, à l'allumage — la mise en route de la console
main()	118 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
Reste	10 o	← main() — le reste d'une division
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>poser()</code> × 4	48 o	sur place, dans <code>main()</code>
<code>image()</code> × 1	3 o	sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <reste>    // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <poser>     // pose une tuile sur une case du fond
```

2. UNE VARIABLE — LENTE

```
uint8_t lente = 0;    // QUELLE lettre montre la lente : 0 = A ... 25 = Z
```

3. UNE VARIABLE — RAPIDE

```
uint8_t rapide = 0;    // QUELLE lettre montre la rapide : 0 = A ... 25 = Z
```

4. UNE VARIABLE — IMAGES

```
uint8_t images = 0;    // UN SEUL chronomètre : il compte de 0 à 59, soit 1 seconde
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

        // attend l'image suivante : 60 tours par seconde

        // Le chronomètre avance d'une image, et repart à 0 après 59 :
        // il fait un tour complet chaque seconde.
        images++;
        if (images == 60) images = 0;

        // La RAPIDE : quand images vaut 0, 15, 30 ou 45 (4 fois par seconde).
        // images % 15 = le reste de images divisé par 15 ;
        // il vaut 0 seulement pour les multiples de 15.
        // (rapide + 1) % sizeof(ALPHABET) : la lettre suivante,
        // et après Z (25 + 1 = 26), 26 % 26 = 0 : retour à A.
        if (images % 15 == 0) rapide = (rapide + 1) % sizeof(ALPHABET);

        // La LENTE : quand le chronomètre repasse par 0 (1 fois par seconde).
        if (images == 0) lente = (lente + 1) % sizeof(ALPHABET);

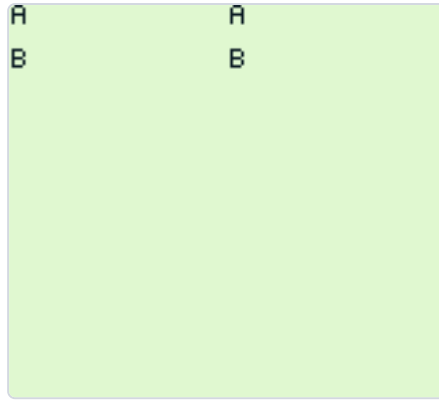
        poser(0, 0, ALPHABET[lente]);    // ligne 0 : la lente
        poser(0, 2, ALPHABET[rapide]);    // ligne 2 : la rapide
        poser(10, 0, ALPHABET[lente]);    // la lente, en colonne 10 (le 0.15.1)
        poser(10, 2, ALPHABET[rapide]);    // la rapide, en colonne 10 (le 0.15.1)
    }
}
```

3. Ce qui se passe, étape par étape

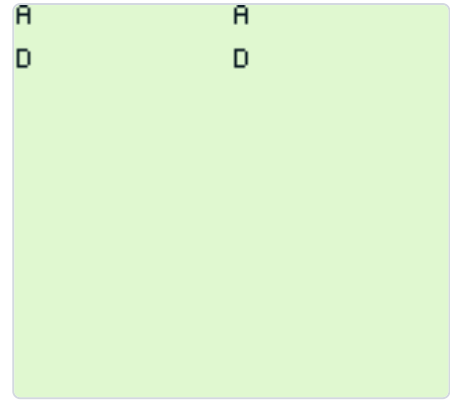
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



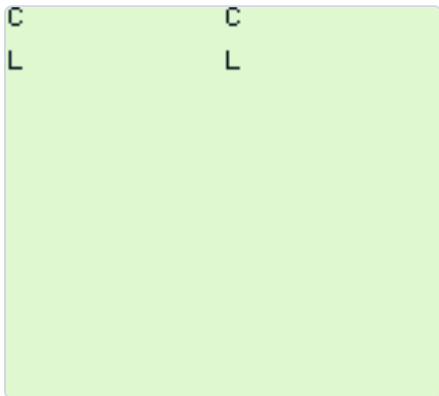
la toute première image — la console vient de démarrer — image 1



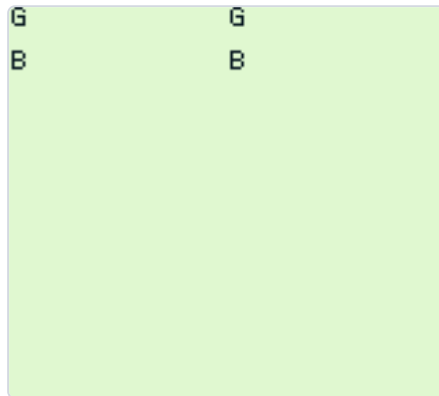
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

La lente et la rapide, en colonne 0 et en colonne 10.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ colonne 0 : la lente (ligne 0) et la rapide (ligne 2) changent
- ✓ colonne 10 : la lente (ligne 0) et la rapide (ligne 2) changent

Ensuite : **0.16. Deux rythmes, écrits en millisecondes** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++